ONLINE LEARNING

QUESTION BANK 3

Pseudocode + OOPs

200 Unique Questions | Detailed Explanations | Speed Tricks & Exam Strategy

Accenture 2027 Preparation Series

Section	Questions	Coverage
Section 1 — Bitwise Operations	Q1–Q30	AND/OR/XOR outputs, shifts, identities, masks, power-of-two test, XOR swap, bit counting
Section 2 — Loops & Conditions Tracing	Q31–Q70	Sum/product loops, WHILE iteration counts, nested totals, BREAK vs CONTINUE, countdowns
Section 3 — Arrays & Strings	Q71–Q110	Traversal, indexing, swap/reverse/rotate, search, substring, character frequency, sorting
Section 4 — Recursion & Functions	Q111–Q130	Factorial family, sum-to-n, Fibonacci, digit-sum, halving counts — full unwinding traces
Section 5 — Operators & Expressions	Q131–Q150	Precedence, DIV/MOD, boolean logic, assignment sequencing, no-temp swaps
Section 6 — OOPs Concepts	Q151–Q200	Four pillars, overloading vs overriding, access modifiers, static, constructors, interfaces, SOLID

How to use this bank: This mirrors the exam's most elimination-heavy material. No pen and paper is allowed in the real test, so practise tracing each snippet mentally — say variable values aloud in your head per line. Every answer here was produced by actually executing the code, and every explanation shows the full trace plus the trap the wrong options set.

SECTION 1 — Bitwise Operations (Q1–Q30)

Q1. What does this print?

```
SET ans = 6 << 3 // LEFT shift by 3
PRINT ans
```

A) 48 **B)** 6 **C)** 50 **D)** 46

Answer: A) 48

Explanation: A left shift by 3 multiplies by 2^3 = 8: 6 × 8 = **48** (binary 00000110 → 00110000). Shifts-as-fast-multiplication/division is the single most tested bitwise idea in this exam.

Q2. What is printed?

```
SET val = 46 AND 29 // bitwise AND
PRINT val
```

A) 51 **B)** 12 **C)** 11 **D)** 75

Answer: B) 12

Explanation: Write both in binary: 46 = 00101110, 29 = 00011101. AND keeps a 1 only where BOTH bits are 1: 00001100 = **12**. Distractors include the OR (63) and XOR (51) results — computing the wrong operator is the designed mistake here.

Q3. The console will print:

```
DECLARE num
num = 32 OR 18 // bitwise OR
OUTPUT num
```

A) 50 **B)** 53 **C)** 47 **D)** 48

Answer: A) 50

Explanation: Binary forms: 32 = 00100000, 18 = 00010010. OR sets a 1 where EITHER bit is 1, giving 00110010 = **50**. OR can never be smaller than the larger operand — a sanity check that instantly kills any option below 32.

Q4. Output of the following?

```
res = 55 XOR 55
PRINT res
```

A) 55 **B)** -1 **C)** 0 **D)** 1

Answer: C) 0

Explanation: Any value XOR itself cancels every bit: identical bits differ nowhere, so the result is **0**. This identity powers the classic 'find the unique element' trick where XOR-ing a whole array leaves only the non-repeated number.

Q5. Predict the output of this snippet:

```
SET a = 15 AND 11 // bitwise AND
PRINT a
```

A) 26 **B)** 11 **C)** 4 **D)** 14

Answer: B) 11

Explanation: Write both in binary: 15 = 00001111, 11 = 00001011. AND keeps a 1 only where BOTH bits are 1: 00001011 = **11**. Distractors include the OR (15) and XOR (4) results — computing the wrong operator is the designed mistake here.

Q6. What does this print?

```
SET val = 21 << 2 // LEFT shift by 2
PRINT val
```

A) 21 **B)** 84 **C)** 168 **D)** 86

Answer: B) 84

Explanation: A left shift by 2 multiplies by 2^2 = 4: 21 × 4 = **84** (binary 00010101 → 01010100). Shifts-as-fast-multiplication/division is the single most tested bitwise idea in this exam.

Q7. What gets displayed?

```
k = 82 OR 0
DISPLAY k
```

A) 83 **B)** 0 **C)** 81 **D)** 82

Answer: D) 82

Explanation: OR with 0 changes nothing — 0 contributes no 1-bits — so the value stays **82**. The parallel identities: AND with 0 gives 0, AND with all-1s gives the number itself. Identity questions are free marks if the three rules are memorised as a set.

Q8. Predict what gets printed:

```
t := 19 XOR 40
WRITE t
```

A) 55 **B)** 62 **C)** 61 **D)** 59

Answer: D) 59

Explanation: 19 = 00010011 and 40 = 00101000; XOR outputs 1 only where the bits DIFFER: 00111011 = **59**. Remember the special cases n XOR n = 0 and n XOR 0 = n — the exam builds several questions on them.

Q9. Three XOR swaps run below. What is the FINAL value of w?

```
z = 26 ; w = 12
z = z XOR w
w = z XOR w
z = z XOR w
PRINT w
```

A) 12 **B)** 29 **C)** 31 **D)** 26

Answer: D) 26

Explanation: The triple-XOR pattern SWAPS two variables without a temp. Trace: after line 2, z holds 26^12 = 22; line 3 makes w = (26^12)^12 = 26; line 4 restores z = 12. So w prints **26**. Recognise the pattern instead of re-deriving it under time pressure.

Q10. After this runs, the output is:

```
q := 27 XOR 53
WRITE q
```

A) 50 **B)** 48 **C)** 42 **D)** 46

Answer: D) 46

Explanation: 27 = 00011011 and 53 = 00110101; XOR outputs 1 only where the bits DIFFER: 00101110 = **46**. Remember the special cases n XOR n = 0 and n XOR 0 = n — the exam builds several questions on them.

Q11. A power-of-two test runs below. What prints?

```
total = 16 AND 15
PRINT total
```

A) 0 **B)** 1 **C)** 16 **D)** 15

Answer: A) 0

Explanation: 16 in binary is a single 1 followed by zeros (00010000), while 15 is all 1s below it (00001111); they share no set bits, so AND gives **0**. n AND (n–1) == 0 is THE standard power-of-two check — memorise the idiom, it appears verbatim in exams.

Q12. Evaluate and choose the output:

```
DECLARE b
b = 42 OR 58 // bitwise OR
OUTPUT b
```

A) 60 **B)** 16 **C)** 58 **D)** 61

Answer: C) 58

Explanation: Binary forms: 42 = 00101010, 58 = 00111010. OR sets a 1 where EITHER bit is 1, giving 00111010 = **58**. OR can never be smaller than the larger operand — a sanity check that instantly kills any option below 58.

Q13. Find the result:

```
DECLARE p
p = 14 OR 55 // bitwise OR
OUTPUT p
```

A) 60 **B)** 61 **C)** 57 **D)** 63

Answer: D) 63

Explanation: Binary forms: 14 = 00001110, 55 = 00110111. OR sets a 1 where EITHER bit is 1, giving 00111111 = **63**. OR can never be smaller than the larger operand — a sanity check that instantly kills any option below 55.

Q14. How many 1-bits does the loop count?

```
n = 13 ; c = 0
WHILE n > 0
  c = c + (n AND 1)
  n = n >> 1
END WHILE
PRINT c
```

A) 2 **B)** 3 **C)** 1 **D)** 5

Answer: B) 3

Explanation: 13 = 00001101 in binary. Each pass adds the lowest bit (n AND 1) and shifts right; the total of 1-bits is **3**. This is the population-count pattern — AND-1 reads a bit, >>1 moves to the next — worth recognising as a unit.

Q15. After execution, the PRINT statement shows:

```
SET k = 10 AND 16 // bitwise AND
PRINT k
```

A) 3 **B)** 0 **C)** -1 **D)** -2

Answer: B) 0

Explanation: Write both in binary: 10 = 00001010, 16 = 00010000. AND keeps a 1 only where BOTH bits are 1: 00000000 = **0**. Distractors include the OR (26) and XOR (26) results — computing the wrong operator is the designed mistake here.

Q16. Choose the value displayed:

```
d := 60 XOR 21
WRITE d
```

A) 41 **B)** 20 **C)** 43 **D)** 37

Answer: A) 41

Explanation: 60 = 00111100 and 21 = 00010101; XOR outputs 1 only where the bits DIFFER: 00101001 = **41**. Remember the special cases n XOR n = 0 and n XOR 0 = n — the exam builds several questions on them.

Q17. What does this print?

```
SET a = 6 >> 3 // RIGHT shift by 3
PRINT a
```

A) 6 **B)** 0 **C)** 2 **D)** 1

Answer: B) 0

Explanation: A right shift by 3 integer-divides by 2^3 = 8: 6 ÷ 8 = **0** (binary 00000110 → 00000000). Shifts-as-fast-multiplication/division is the single most tested bitwise idea in this exam.

Q18. Result of execution:

```
y := 46 XOR 28
WRITE y
```

A) 12 **B)** 48 **C)** 50 **D)** 62

Answer: C) 50

Explanation: 46 = 00101110 and 28 = 00011100; XOR outputs 1 only where the bits DIFFER: 00110010 = **50**. Remember the special cases n XOR n = 0 and n XOR 0 = n — the exam builds several questions on them.

Q19. The snippet prints:

```
x := 41 XOR 37
WRITE x
```

A) 16 **B)** 33 **C)** 8 **D)** 12

Answer: D) 12

Explanation: 41 = 00101001 and 37 = 00100101; XOR outputs 1 only where the bits DIFFER: 00001100 = **12**. Remember the special cases n XOR n = 0 and n XOR 0 = n — the exam builds several questions on them.

Q20. A mask extracts part of the number. Output?

```
x = 155
PRINT x AND 15 // 15 = 00001111
```

A) 11 **B)** 13 **C)** 8 **D)** 9

Answer: A) 11

Explanation: AND-ing with 15 (binary 1111) keeps only the lowest 4 bits of 155 (10011011): 1011 = **11** — equivalently 155 MOD 16. Masks are 'bit filters'; AND with a mask zeroes everything outside the mask's 1s.

Q21. What does the display show after running this?

```
DECLARE s
s = 41 OR 49 // bitwise OR
OUTPUT s
```

A) 24 **B)** 55 **C)** 57 **D)** 60

Answer: C) 57

Explanation: Binary forms: 41 = 00101001, 49 = 00110001. OR sets a 1 where EITHER bit is 1, giving 00111001 = **57**. OR can never be smaller than the larger operand — a sanity check that instantly kills any option below 49.

Q22. The following pseudocode displays:

```
SET res = 7 AND 16 // bitwise AND
PRINT res
```

A) 0 **B)** 3 **C)** -1 **D)** 23

Answer: A) 0

Explanation: Write both in binary: 7 = 00000111, 16 = 00010000. AND keeps a 1 only where BOTH bits are 1: 00000000 = **0**. Distractors include the OR (23) and XOR (23) results — computing the wrong operator is the designed mistake here.

Q23. Trace the code and pick the output:

```
SET m = 42 AND 8 // bitwise AND
PRINT m
```

A) 50 **B)** 8 **C)** 34 **D)** 5

Answer: B) 8

Explanation: Write both in binary: 42 = 00101010, 8 = 00001000. AND keeps a 1 only where BOTH bits are 1: 00001000 = **8**. Distractors include the OR (42) and XOR (34) results — computing the wrong operator is the designed mistake here.

Q24. Predict the output:

```
m = 61 AND 1
PRINT m
```

A) 1 **B)** 0 **C)** 61 **D)** Error

Answer: A) 1

Explanation: AND-ing with 1 isolates the LAST bit. 61 is odd, so its last binary digit is 1, and the result is **1**. This is the standard odd/even test in bit form: (n AND 1) = 1 means odd, 0 means even — expect it inside larger loop questions too.

Q25. This code outputs:

```
DECLARE total
total = 19 OR 12 // bitwise OR
OUTPUT total
```

A) 29 **B)** 35 **C)** 31 **D)** 33

Answer: C) 31

Explanation: Binary forms: 19 = 00010011, 12 = 00001100. OR sets a 1 where EITHER bit is 1, giving 00011111 = **31**. OR can never be smaller than the larger operand — a sanity check that instantly kills any option below 19.

Q26. What value appears on screen?

```
SET z = 32 AND 15 // bitwise AND
PRINT z
```

A) 47 **B)** 2 **C)** -2 **D)** 0

Answer: D) 0

Explanation: Write both in binary: 32 = 00100000, 15 = 00001111. AND keeps a 1 only where BOTH bits are 1: 00000000 = **0**. Distractors include the OR (47) and XOR (47) results — computing the wrong operator is the designed mistake here.

Q27. What does this print?

```
SET c = 5 >> 2 // RIGHT shift by 2
PRINT c
```

A) 1 **B)** 3 **C)** 2 **D)** 5

Answer: A) 1

Explanation: A right shift by 2 integer-divides by 2^2 = 4: 5 ÷ 4 = **1** (binary 00000101 → 00000001). Shifts-as-fast-multiplication/division is the single most tested bitwise idea in this exam.

Q28. Determine the printed value:

```
DECLARE w
w = 17 OR 46 // bitwise OR
OUTPUT w
```

A) 65 **B)** 60 **C)** 67 **D)** 63

Answer: D) 63

Explanation: Binary forms: 17 = 00010001, 46 = 00101110. OR sets a 1 where EITHER bit is 1, giving 00111111 = **63**. OR can never be smaller than the larger operand — a sanity check that instantly kills any option below 46.

Q29. Setting a specific bit with OR — what prints?

```
x = 12
x = x OR 2 // 2 = 00000010
PRINT x
```

A) 12 **B)** 18 **C)** 16 **D)** 14

Answer: D) 14

Explanation: OR with 2 forces bit-1 (value 2) ON while leaving other bits untouched: 00001100 OR 00000010 = 00001110 = **14**. OR-to-set, AND-to-clear, XOR-to-toggle: the three bit-manipulation verbs in one line.

Q30. What is the output?

```
cnt := 40 XOR 14
WRITE cnt
```

A) 34 **B)** 38 **C)** 40 **D)** 42

Answer: B) 38

Explanation: 40 = 00101000 and 14 = 00001110; XOR outputs 1 only where the bits DIFFER: 00100110 = **38**. Remember the special cases n XOR n = 0 and n XOR 0 = n — the exam builds several questions on them.

SECTION 2 — Loops & Conditions Tracing (Q31–Q70)

Q31. What is the final value of b?

```
b = 0
FOR i = 1 TO 5
  FOR j = 1 TO i
    b = b + 1
  END FOR
END FOR
PRINT b
```

A) 22 **B)** 13 **C)** 17 **D)** 15

Answer: D) 15

Explanation: Count body executions: the inner count grows with i: 1 + 2 + ... + 5 = 5×6/2 = **15**. Whether the inner bound is fixed, 'TO i', or 'FROM i' completely changes the total — read the inner header before anything else.

Q32. Find the number of iterations performed:

```
val = 3 ; a = 0
WHILE val < 50
  val = val * 2
  a = a + 1
END WHILE
PRINT a
```

A) 6 **B)** 5 **C)** 3 **D)** 7

Answer: B) 5

Explanation: val takes the values 3 → 6 → 12 → 24 → 48 → 96; the body runs once per arrow, i.e. **5** times, stopping when 96 is no longer < 50. Doubling/tripling loops are logarithmic: the count is roughly log of the limit — sanity-check with that.

Q33. What is the final value of cnt?

```
cnt = 0
FOR i = 1 TO 6
  FOR j = 1 TO i
    cnt = cnt + 1
  END FOR
END FOR
PRINT cnt
```

A) 27 **B)** 15 **C)** 30 **D)** 21

Answer: D) 21

Explanation: Count body executions: the inner count grows with i: 1 + 2 + ... + 6 = 6×7/2 = **21**. Whether the inner bound is fixed, 'TO i', or 'FROM i' completely changes the total — read the inner header before anything else.

Q34. The PRINT shows:

```
ans = 0
FOR i = 1 TO 10
  IF i == 4 THEN BREAK
  ans = ans + i
END FOR
PRINT ans
```

A) 10 **B)** 5 **C)** 6 **D)** 7

Answer: C) 6

Explanation: The BREAK fires when i reaches 4, BEFORE that iteration's addition — so only 1..3 are summed: 1 + 2 + 3 = **6**. BREAK exits the whole loop instantly; CONTINUE would merely skip one turn — don't conflate them.

Q35. The code prints:

```
b = 1
FOR i = 1 TO 7
  b = b * i
END FOR
PRINT b
```

A) 5050 **B)** 2520 **C)** 7560 **D)** 5040

Answer: D) 5040

Explanation: This multiplies 1 by every i from 1 to 7: 1×1×2×3×4×5×6×7 = **5040** — exactly 7! (factorial). Spot factorial loops by the pattern result = result * i.

Q36. Give the output of this snippet:

```
t = 2
FOR i = 1 TO 7
  t = t * i
END FOR
PRINT t
```

A) 10090 **B)** 10087 **C)** 10080 **D)** 5040

Answer: C) 10080

Explanation: This multiplies 2 by every i from 1 to 7: 2×1×2×3×4×5×6×7 = **10080** — twice 7! because of the initial value 2. Spot factorial loops by the pattern result = result * i.

Q37. What does the code output?

```
res = 0
FOR i = 1 TO 13
  IF i MOD 3 == 0 THEN CONTINUE
  res = res + i
END FOR
PRINT res
```

A) 58 **B)** 91 **C)** 61 **D)** 64

Answer: C) 61

Explanation: CONTINUE skips every multiple of 3, so the added values are 1, 2, 4, 5, 7, 8, 10, 11, 13, totalling **61**. MOD 3 == 0 identifies multiples — flip the comparison to != and the kept/skipped sets swap.

Q38. What FINAL value of s is printed?

```
s = 29
WHILE s > 0
  s = s - 2
END WHILE
PRINT s
```

A) -1 **B)** -2 **C)** 1 **D)** 0

Answer: A) -1

Explanation: Repeated subtraction of 2 from 29 continues while the value exceeds 0; after 15 passes it lands on **-1**, the first value NOT greater than 0. Final value = 29 – 15×2; compute it directly instead of stepping one by one.

Q39. How many passes through the loop occur?

```
m = 3 ; z = 0
WHILE m < 200
  m = m * 2
  z = z + 1
END WHILE
PRINT z
```

A) 9 **B)** 7 **C)** 5 **D)** 6

Answer: B) 7

Explanation: m takes the values 3 → 6 → 12 → 24 → 48 → 96 → 192 → 384; the body runs once per arrow, i.e. **7** times, stopping when 384 is no longer < 200. A common wrong answer counts the values (8) instead of the transitions (7) — arrows, not nodes.

Q40. What count does c hold when the loop ends?

```
ans = 2 ; c = 0
WHILE ans < 50
  ans = ans * 2
  c = c + 1
END WHILE
PRINT c
```

A) 4 **B)** 5 **C)** 7 **D)** 3

Answer: B) 5

Explanation: ans takes the values 2 → 4 → 8 → 16 → 32 → 64; the body runs once per arrow, i.e. **5** times, stopping when 64 is no longer < 50. Note the loop tests BEFORE multiplying, so the pass that pushes past 50 still counts.

Q41. What appears on the console?

```
d = 0
FOR i = 2 TO 7 STEP 1
  d = d + i
END FOR
PRINT d
```

A) 27 **B)** 25 **C)** 29 **D)** 23

Answer: A) 27

Explanation: The loop visits i = 2, 3, 4, 5, 6, 7 — 6 iteration(s). Adding them: 2 + 3 + 4 + 5 + 6 + 7 = **27**. List actual i-values before summing; guessing the range breeds off-by-ones.

Q42. What does the code output?

```
a = 0
FOR i = 1 TO 12
  IF i MOD 2 == 0 THEN CONTINUE
  a = a + i
END FOR
PRINT a
```

A) 78 **B)** 39 **C)** 36 **D)** 38

Answer: C) 36

Explanation: CONTINUE skips every multiple of 2, so the added values are 1, 3, 5, 7, 9, 11, totalling **36**. The complementary sum (only multiples of 2) is 42 — sitting among the options.

Q43. Output?

```
y = 0
FOR i = 1 TO 11
  IF i == 7 THEN BREAK
  y = y + i
END FOR
PRINT y
```

A) 22 **B)** 21 **C)** 66 **D)** 14

Answer: B) 21

Explanation: The BREAK fires when i reaches 7, BEFORE that iteration's addition — so only 1..6 are summed: 1 + 2 + 3 + 4 + 5 + 6 = **21**. The loop's stated limit 11 is irrelevant once BREAK triggers — a decoy bound.

Q44. Output?

```
cnt = 2
FOR i = 1 TO 6
  cnt = cnt * i
END FOR
PRINT cnt
```

A) 1450 **B)** 1446 **C)** 720 **D)** 1440

Answer: D) 1440

Explanation: This multiplies 2 by every i from 1 to 6: 2×1×2×3×4×5×6 = **1440** — twice 6! because of the initial value 2. Spot factorial loops by the pattern result = result * i.

Q45. Output?

```
p = 0
FOR i = 3 TO 14 STEP 2
  p = p + i
END FOR
PRINT p
```

A) 51 **B)** 46 **C)** 48 **D)** 50

Answer: C) 48

Explanation: The loop visits i = 3, 5, 7, 9, 11, 13 — 6 iteration(s). Adding them: 3 + 5 + 7 + 9 + 11 + 13 = **48**. List actual i-values before summing; guessing the range breeds off-by-ones.

Q46. After the loop ends, the output is:

```
cnt = 0
FOR i = 3 TO 7 STEP 2
  cnt = cnt + i
END FOR
PRINT cnt
```

A) 15 **B)** 18 **C)** 11 **D)** 13

Answer: A) 15

Explanation: The loop visits i = 3, 5, 7 — 3 iteration(s). Adding them: 3 + 5 + 7 = **15**. List actual i-values before summing; guessing the range breeds off-by-ones.

Q47. What FINAL value of w is printed?

```
w = 18
WHILE w > 4
  w = w - 3
END WHILE
PRINT w
```

A) 3 **B)** 0 **C)** 6 **D)** 4

Answer: A) 3

Explanation: Repeated subtraction of 3 from 18 continues while the value exceeds 4; after 5 passes it lands on **3**, the first value NOT greater than 4. Overshoot below the limit is legal — the condition is checked BEFORE each subtraction, not after.

Q48. After the loop ends, the output is:

```
b = 0
FOR i = 3 TO 7 STEP 1
  b = b + i
END FOR
PRINT b
```

A) 22 **B)** 25 **C)** 28 **D)** 24

Answer: B) 25

Explanation: The loop visits i = 3, 4, 5, 6, 7 — 5 iteration(s). Adding them: 3 + 4 + 5 + 6 + 7 = **25**. STEP 1 skips values — including a skipped value is the planted error.

Q49. What appears on the console?

```
x = 0
FOR i = 1 TO 10
  IF i == 7 THEN BREAK
  x = x + i
END FOR
PRINT x
```

A) 21 **B)** 55 **C)** 14 **D)** 28

Answer: A) 21

Explanation: The BREAK fires when i reaches 7, BEFORE that iteration's addition — so only 1..6 are summed: 1 + 2 + 3 + 4 + 5 + 6 = **21**. Move the IF below the addition and 7 would join the sum (28) — position is everything.

Q50. What is the final value of t?

```
t = 0
FOR i = 1 TO 6
  FOR j = i TO 6
    t = t + 1
  END FOR
END FOR
PRINT t
```

A) 15 **B)** 33 **C)** 21 **D)** 27

Answer: C) 21

Explanation: Count body executions: the inner loop runs (b – i + 1) times per i: 6 + 5 + 4 + 3 + 2 + 1 = **21**. Whether the inner bound is fixed, 'TO i', or 'FROM i' completely changes the total — read the inner header before anything else.

Q51. What does the code output?

```
val = 0
FOR i = 1 TO 10
  IF i MOD 2 == 0 THEN CONTINUE
  val = val + i
END FOR
PRINT val
```

A) 28 **B)** 23 **C)** 55 **D)** 25

Answer: D) 25

Explanation: CONTINUE skips every multiple of 2, so the added values are 1, 3, 5, 7, 9, totalling **25**. CONTINUE abandons just the current pass; the loop itself marches on to the next i.

Q52. What is the final value of num?

```
num = 0
FOR i = 1 TO 5
  FOR j = i TO 6
    num = num + 1
  END FOR
END FOR
PRINT num
```

A) 31 **B)** 25 **C)** 15 **D)** 20

Answer: D) 20

Explanation: Count body executions: the inner loop runs (b – i + 1) times per i: 6 + 5 + 4 + 3 + 2 = **20**. Whether the inner bound is fixed, 'TO i', or 'FROM i' completely changes the total — read the inner header before anything else.

Q53. What is the final value of s?

```
s = 0
FOR i = 1 TO 5
  FOR j = 1 TO 6
    s = s + 1
  END FOR
END FOR
PRINT s
```

A) 36 **B)** 41 **C)** 30 **D)** 24

Answer: C) 30

Explanation: Count body executions: the inner loop runs 6 times for EACH of the 5 outer iterations: 5 × 6 = **30**. Whether the inner bound is fixed, 'TO i', or 'FROM i' completely changes the total — read the inner header before anything else.

Q54. What FINAL value of z is printed?

```
z = 24
WHILE z > 5
  z = z - 4
END WHILE
PRINT z
```

A) 4 **B)** 3 **C)** 6 **D)** 8

Answer: A) 4

Explanation: Repeated subtraction of 4 from 24 continues while the value exceeds 5; after 5 passes it lands on **4**, the first value NOT greater than 5. Final value = 24 – 5×4; compute it directly instead of stepping one by one.

Q55. What does the code output?

```
k = 0
FOR i = 1 TO 12
  IF i MOD 3 == 0 THEN CONTINUE
  k = k + i
END FOR
PRINT k
```

A) 51 **B)** 48 **C)** 78 **D)** 45

Answer: B) 48

Explanation: CONTINUE skips every multiple of 3, so the added values are 1, 2, 4, 5, 7, 8, 10, 11, totalling **48**. MOD 3 == 0 identifies multiples — flip the comparison to != and the kept/skipped sets swap.

Q56. The final printed value of k is:

```
res = 2 ; k = 0
WHILE res < 500
  res = res * 2
  k = k + 1
END WHILE
PRINT k
```

A) 9 **B)** 7 **C)** 8 **D)** 6

Answer: C) 8

Explanation: res takes the values 2 → 4 → 8 → 16 → 32 → 64 → 128 → 256 → 512; the body runs once per arrow, i.e. **8** times, stopping when 512 is no longer < 500. List the value chain first; the answer is simply the number of arrows in it.

Q57. What is the final value printed?

```
q = 0
FOR i = 5 TO 14 STEP 2
  q = q + i
END FOR
PRINT q
```

A) 38 **B)** 45 **C)** 59 **D)** 43

Answer: B) 45

Explanation: The loop visits i = 5, 7, 9, 11, 13 — 5 iteration(s). Adding them: 5 + 7 + 9 + 11 + 13 = **45**. With step 2, the count of terms is (last−first)/step + 1 = 5 — verify counts that way.

Q58. What iteration count is displayed?

```
w = 1 ; total = 0
WHILE w < 500
  w = w * 3
  total = total + 1
END WHILE
PRINT total
```

A) 8 **B)** 4 **C)** 5 **D)** 6

Answer: D) 6

Explanation: w takes the values 1 → 3 → 9 → 27 → 81 → 243 → 729; the body runs once per arrow, i.e. **6** times, stopping when 729 is no longer < 500. When the multiplier is 3, growth is even steeper — expect a smaller count than a doubling loop.

Q59. Output?

```
s = 0
FOR i = 3 TO 13 STEP 3
  s = s + i
END FOR
PRINT s
```

A) 30 **B)** 23 **C)** 43 **D)** 33

Answer: A) 30

Explanation: The loop visits i = 3, 6, 9, 12 — 4 iteration(s). Adding them: 3 + 6 + 9 + 12 = **30**. Check whether 13 itself is reachable with this step — here it is not.

Q60. What is the final value of p?

```
p = 0
FOR i = 1 TO 3
  FOR j = 1 TO 4
    p = p + 1
  END FOR
END FOR
PRINT p
```

A) 19 **B)** 15 **C)** 9 **D)** 12

Answer: D) 12

Explanation: Count body executions: the inner loop runs 4 times for EACH of the 3 outer iterations: 3 × 4 = **12**. Whether the inner bound is fixed, 'TO i', or 'FROM i' completely changes the total — read the inner header before anything else.

Q61. How many times does the loop body execute?

```
x = 1 ; y = 0
WHILE x < 200
  x = x * 3
  y = y + 1
END WHILE
PRINT y
```

A) 4 **B)** 3 **C)** 7 **D)** 5

Answer: D) 5

Explanation: x takes the values 1 → 3 → 9 → 27 → 81 → 243; the body runs once per arrow, i.e. **5** times, stopping when 243 is no longer < 200. Exponential growth finishes in a handful of steps — count the multiplications, don't simulate endlessly.

Q62. The code prints:

```
num = 0
FOR i = 5 TO 15 STEP 3
  num = num + i
END FOR
PRINT num
```

A) 33 **B)** 35 **C)** 38 **D)** 43

Answer: C) 38

Explanation: The loop visits i = 5, 8, 11, 14 — 4 iteration(s). Adding them: 5 + 8 + 11 + 14 = **38**. The TO bound is INCLUSIVE; dropping the last term is the classic slip.

Q63. What does the code output?

```
c = 0
FOR i = 1 TO 8
  IF i MOD 2 == 0 THEN CONTINUE
  c = c + i
END FOR
PRINT c
```

A) 16 **B)** 14 **C)** 36 **D)** 19

Answer: A) 16

Explanation: CONTINUE skips every multiple of 2, so the added values are 1, 3, 5, 7, totalling **16**. The complementary sum (only multiples of 2) is 20 — sitting among the options.

Q64. What appears on the console?

```
q = 0
FOR i = 1 TO 10
  IF i == 5 THEN BREAK
  q = q + i
END FOR
PRINT q
```

A) 5 **B)** 15 **C)** 10 **D)** 11

Answer: C) 10

Explanation: The BREAK fires when i reaches 5, BEFORE that iteration's addition — so only 1..4 are summed: 1 + 2 + 3 + 4 = **10**. Move the IF below the addition and 5 would join the sum (15) — position is everything.

Q65. What FINAL value of m is printed?

```
m = 27
WHILE m > 5
  m = m - 3
END WHILE
PRINT m
```

A) 5 **B)** 3 **C)** 2 **D)** 0

Answer: B) 3

Explanation: Repeated subtraction of 3 from 27 continues while the value exceeds 5; after 8 passes it lands on **3**, the first value NOT greater than 5. Final value = 27 – 8×3; compute it directly instead of stepping one by one.

Q66. What appears on the console?

```
p = 2
FOR i = 1 TO 4
  p = p * i
END FOR
PRINT p
```

A) 48 **B)** 58 **C)** 24 **D)** 44

Answer: A) 48

Explanation: This multiplies 2 by every i from 1 to 4: 2×1×2×3×4 = **48** — twice 4! because of the initial value 2. Spot factorial loops by the pattern result = result * i.

Q67. What FINAL value of total is printed?

```
total = 24
WHILE total > 1
  total = total - 2
END WHILE
PRINT total
```

A) 0 **B)** -2 **C)** -1 **D)** 2

Answer: A) 0

Explanation: Repeated subtraction of 2 from 24 continues while the value exceeds 1; after 12 passes it lands on **0**, the first value NOT greater than 1. Final value = 24 – 12×2; compute it directly instead of stepping one by one.

Q68. Trace and give the output:

```
t = 0
FOR i = 3 TO 10 STEP 1
  t = t + i
END FOR
PRINT t
```

A) 52 **B)** 47 **C)** 53 **D)** 49

Answer: A) 52

Explanation: The loop visits i = 3, 4, 5, 6, 7, 8, 9, 10 — 8 iteration(s). Adding them: 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 = **52**. Small ranges are traps for overconfidence — trace them fully anyway.

Q69. The code prints:

```
d = 0
FOR i = 1 TO 15
  IF i == 7 THEN BREAK
  d = d + i
END FOR
PRINT d
```

A) 28 **B)** 21 **C)** 22 **D)** 20

Answer: B) 21

Explanation: The BREAK fires when i reaches 7, BEFORE that iteration's addition — so only 1..6 are summed: 1 + 2 + 3 + 4 + 5 + 6 = **21**. Move the IF below the addition and 7 would join the sum (28) — position is everything.

Q70. What is the final value printed?

```
num = 1
FOR i = 1 TO 6
  num = num * i
END FOR
PRINT num
```

A) 1080 **B)** 720 **C)** 726 **D)** 360

Answer: B) 720

Explanation: This multiplies 1 by every i from 1 to 6: 1×1×2×3×4×5×6 = **720** — exactly 6! (factorial). Spot factorial loops by the pattern result = result * i.

SECTION 3 — Arrays & Strings (Q71–Q110)

Q71. The loop skips alternate positions. Output?

```
arr = [39, 30, 25, 65, 59, 54]
t = 0
FOR i = 0 TO 5 STEP 2
  t = t + arr[i]
END FOR
PRINT t
```

A) 118 **B)** 128 **C)** 120 **D)** 123

Answer: D) 123

Explanation: STEP 2 from 0 touches indices 0, 2, 4 — values 39, 25, 59 — totalling **123**. Starting the same loop at 1 would instead pick 30, 65, 54 (sum 149), the mirrored distractor.

Q72. Given the array below (index starts at 0), what is printed?

```
marks = [37, 28, 24, 8, 43]
cnt = 0
FOR i = 0 TO 4
  IF marks[i] > 28 THEN cnt = cnt + 1
END FOR
PRINT cnt
```

A) 4 **B)** -1 **C)** 3 **D)** 2

Answer: D) 2

Explanation: The loop counts elements strictly greater than 28: elements > 28 are 37, 43 → **2**. Confirm the loop covers indices 0..4 (all 5 elements) — off-by-one bounds are the favourite trap in traversal questions.

Q73. The loop skips alternate positions. Output?

```
marks = [13, 38, 60, 36, 18, 89]
t = 0
FOR i = 0 TO 5 STEP 2
  t = t + marks[i]
END FOR
PRINT t
```

A) 96 **B)** 86 **C)** 91 **D)** 88

Answer: C) 91

Explanation: STEP 2 from 0 touches indices 0, 2, 4 — values 13, 60, 18 — totalling **91**. Starting the same loop at 1 would instead pick 38, 36, 89 (sum 163), the mirrored distractor.

Q74. What is the output?

```
data = [67, 87, 61, 66, 68]
temp = data[0]
data[0] = data[2]
data[2] = temp
PRINT data[0]
```

A) 64 **B)** 56 **C)** 61 **D)** 67

Answer: C) 61

Explanation: The three temp lines SWAP positions 0 and 2. After swapping, index 0 holds **61** (originally at index 2). The temp-variable swap is the standard idiom — recognise it as a unit rather than tracing each line every time.

Q75. Given the array below (index starts at 0), what is printed?

```
vals = [48, 44, 31, 27, 47, 9]
cnt = 0
FOR i = 0 TO 5
  IF vals[i] > 44 THEN cnt = cnt + 1
END FOR
PRINT cnt
```

A) 2 **B)** -1 **C)** 5 **D)** 1

Answer: A) 2

Explanation: The loop counts elements strictly greater than 44: elements > 44 are 48, 47 → **2**. Confirm the loop covers indices 0..5 (all 6 elements) — off-by-one bounds are the favourite trap in traversal questions.

Q76. What does this display?

```
s = "BINARY"
PRINT LENGTH(s)
```

A) 6 **B)** 5 **C)** 4 **D)** 8

Answer: A) 6

Explanation: LENGTH counts every character: BINARY has **6** letters. Pair this with loops — 'FOR i = 0 TO LENGTH–1' visits all characters, and forgetting the –1 walks past the end, a bug the exam frequently encodes as an off-by-one option.

Q77. After sorting ascending, which value sits at the given index?

```
items = [78, 74, 49, 35, 18, 73]
SORT items in ascending order
PRINT items[3]
```

A) 35 **B)** 69 **C)** 75 **D)** 73

Answer: D) 73

Explanation: Sorted: [18, 35, 49, 73, 74, 78]. Index 3 then holds **73** — the 4th smallest value. Post-sort index questions are rank questions in disguise; sort first, count second.

Q78. A linear search runs below. What INDEX is printed?

```
list = [33, 25, 19, 23, 87, 81]
FOR i = 0 TO 5
  IF list[i] == 33 THEN
    PRINT i
    BREAK
  END IF
END FOR
```

A) 0 **B)** -1 **C)** 1 **D)** -2

Answer: A) 0

Explanation: 33 first appears at position **0** (zero-based). Linear search prints the INDEX, not the value — questions swap those two constantly, and 'position counting from 1' supplies the other classic wrong option, 1.

Q79. What does the code print?

```
vals = [86, 93, 47, 28, 15, 20]
n = 6
PRINT vals[0] + vals[n - 1]
```

A) 103 **B)** 108 **C)** 100 **D)** 106

Answer: D) 106

Explanation: vals[0] is the first element (86) and vals[n–1] = vals[5] the last (20); together **106**. Writing vals[n] instead of vals[n–1] would run past the array — the out-of-bounds error this idiom exists to avoid.

Q80. What does the code print?

```
list = [41, 8, 28, 17, 81, 34]
n = 6
PRINT list[0] + list[n - 1]
```

A) 89 **B)** 75 **C)** 81 **D)** 78

Answer: B) 75

Explanation: list[0] is the first element (41) and list[n–1] = list[5] the last (34); together **75**. Writing list[n] instead of list[n–1] would run past the array — the out-of-bounds error this idiom exists to avoid.

Q81. Given the array below (index starts at 0), what is printed?

```
data = [8, 55, 36, 22, 4]
small = data[0]
FOR i = 0 TO 4
  IF data[i] < small THEN small = data[i]
END FOR
PRINT small
```

A) 7 **B)** 4 **C)** 6 **D)** 3

Answer: B) 4

Explanation: The loop keeps the smallest element seen so far: comparisons leave the minimum, 4 → **4**. Confirm the loop covers indices 0..4 (all 5 elements) — off-by-one bounds are the favourite trap in traversal questions.

Q82. What total is printed?

```
data = [57, 60, 75, 42, 47, 19]
t = 0
FOR i = 0 TO 5
  IF data[i] MOD 2 == 0 THEN t = t + data[i]
END FOR
PRINT t
```

A) 300 **B)** 106 **C)** 102 **D)** 104

Answer: C) 102

Explanation: MOD 2 == 0 selects the even values: 60, 42 → sum **102**. The odd-sum (198) is planted among the options; check which parity the condition keeps before adding anything.

Q83. After reversing the string, which character is at the given index?

```
s = "VARIABLE"
s = REVERSE(s)
PRINT s[0]
```

A) L **B)** E **C)** B **D)** V

Answer: B) E

Explanation: Reversed, "VARIABLE" becomes "ELBAIRAV"; position 0 of the reversal is **'E'**. Formula alternative: reversed[k] equals original[LENGTH–1–k] = original[7] = 'E'. Two routes to the same letter — use whichever is faster for you.

Q84. What is the output?

```
nums = [15, 14, 75, 17, 44]
temp = nums[0]
nums[0] = nums[2]
nums[2] = temp
PRINT nums[0]
```

A) 75 **B)** 78 **C)** 72 **D)** 70

Answer: A) 75

Explanation: The three temp lines SWAP positions 0 and 2. After swapping, index 0 holds **75** (originally at index 2). The temp-variable swap is the standard idiom — recognise it as a unit rather than tracing each line every time.

Q85. Predict the printed element:

```
data = [29, 89, 41, 54, 62]
REVERSE the array in place
PRINT data[3]
```

A) 91 **B)** 87 **C)** 89 **D)** 93

Answer: C) 89

Explanation: Reversed, the array becomes [62, 54, 41, 89, 29]; index 3 now holds **89**, which originally sat at index 1. Shortcut: after reversal, new index k maps to old index (length – 1 – k) — apply the formula instead of rewriting the array.

Q86. A linear search runs below. What INDEX is printed?

```
scores = [60, 45, 11, 42, 6, 7]
FOR i = 0 TO 5
  IF scores[i] == 7 THEN
    PRINT i
    BREAK
  END IF
END FOR
```

A) 6 **B)** 3 **C)** 7 **D)** 5

Answer: D) 5

Explanation: 7 first appears at position **5** (zero-based). Linear search prints the INDEX, not the value — questions swap those two constantly, and 'position counting from 1' supplies the other classic wrong option, 6.

Q87. What substring is printed?

```
s = "CAMPUS"
PRINT SUBSTRING(s, 1, 2) // start 1, end 2 EXCLUSIVE
```

A) M **B)** CA **C)** AM **D)** A

Answer: D) A

Explanation: With an exclusive end, characters at indices 1..1 are taken: **"A"** (1 characters). The inclusive-end reading gives "AM" — sitting among the options, as always. Inclusive vs exclusive end is the entire battle in substring questions.

Q88. Predict the printed element:

```
vals = [87, 28, 86, 82, 33]
ROTATE LEFT by 1 position(s)
PRINT vals[2]
```

A) 82 **B)** 78 **C)** 86 **D)** 84

Answer: A) 82

Explanation: A left rotation by 1 moves the first 1 element(s) to the back: [28, 86, 82, 33, 87]. Index 2 then holds **82**. Mapping rule: new index k takes the old element at (k + 1) mod 5 — modular thinking beats rewriting under time pressure.

Q89. What is the output?

```
marks = [41, 57, 66, 24, 25]
PRINT marks[1] + marks[0]
```

A) 98 **B)** 95 **C)** 57 **D)** 101

Answer: A) 98

Explanation: Zero-based indexing: marks[1] = 57 and marks[0] = 41; their sum is **98**. Index n is the (n+1)th element — reciting that rule before answering prevents the shifted-by-one errors the options bait.

Q90. Given the array below (index starts at 0), what is printed?

```
list = [25, 58, 37, 44, 30, 21]
total = 0
FOR i = 0 TO 5
  total = total + list[i]
END FOR
PRINT total
```

A) 217 **B)** 216 **C)** 213 **D)** 215

Answer: D) 215

Explanation: The loop adds every element: 25+58+37+44+30+21 → **215**. Confirm the loop covers indices 0..5 (all 6 elements) — off-by-one bounds are the favourite trap in traversal questions.

Q91. How many times does 'R' occur?

```
s = "PROGRAM"
cnt = 0
FOR each ch IN s
  IF ch == 'R' THEN cnt = cnt + 1
END FOR
PRINT cnt
```

A) 4 **B)** 3 **C)** 1 **D)** 2

Answer: D) 2

Explanation: Scan "PROGRAM" for 'R': it appears **2** time(s). Frequency loops compare each character to the target and increment a counter — mark the letters on the word as you count, because double-counting or skipping one letter is how the neighbouring options win.

Q92. Predict the printed element:

```
scores = [22, 56, 33, 53, 48]
ROTATE LEFT by 2 position(s)
PRINT scores[2]
```

A) 50 **B)** 33 **C)** 46 **D)** 48

Answer: D) 48

Explanation: A left rotation by 2 moves the first 2 element(s) to the back: [33, 53, 48, 22, 56]. Index 2 then holds **48**. Mapping rule: new index k takes the old element at (k + 2) mod 5 — modular thinking beats rewriting under time pressure.

Q93. What total is printed?

```
nums = [35, 49, 78, 80, 41, 65]
t = 0
FOR i = 0 TO 5
  IF nums[i] MOD 2 == 0 THEN t = t + nums[i]
END FOR
PRINT t
```

A) 154 **B)** 158 **C)** 348 **D)** 156

Answer: B) 158

Explanation: MOD 2 == 0 selects the even values: 78, 80 → sum **158**. The odd-sum (190) is planted among the options; check which parity the condition keeps before adding anything.

Q94. Predict the printed element:

```
arr = [30, 70, 42, 38, 13]
REVERSE the array in place
PRINT arr[2]
```

A) 40 **B)** 44 **C)** 46 **D)** 42

Answer: D) 42

Explanation: Reversed, the array becomes [13, 38, 42, 70, 30]; index 2 now holds **42**, which originally sat at index 2. Shortcut: after reversal, new index k maps to old index (length – 1 – k) — apply the formula instead of rewriting the array.

Q95. After reversing the string, which character is at the given index?

```
s = "ENGINEER"
s = REVERSE(s)
PRINT s[1]
```

A) G **B)** E **C)** R **D)** N

Answer: B) E

Explanation: Reversed, "ENGINEER" becomes "REENIGNE"; position 1 of the reversal is **'E'**. Formula alternative: reversed[k] equals original[LENGTH–1–k] = original[6] = 'E'. Two routes to the same letter — use whichever is faster for you.

Q96. Which character prints?

```
s = "NETWORK"
PRINT s[1] // 0-based index
```

A) T **B)** O **C)** E **D)** N

Answer: C) E

Explanation: Counting from 0: 0=N, 1=E. So s[1] is **'E'**. Most wrong answers here come from starting the count at 1 — the neighbouring letters are planted as options for exactly that slip.

Q97. Given the array below (index starts at 0), what is printed?

```
scores = [7, 21, 41, 23, 30, 33]
big = scores[0]
FOR i = 0 TO 5
  IF scores[i] > big THEN big = scores[i]
END FOR
PRINT big
```

A) 44 **B)** 41 **C)** 43 **D)** 38

Answer: B) 41

Explanation: The loop tracks the largest element seen so far: comparisons leave the maximum, 41 → **41**. Confirm the loop covers indices 0..5 (all 6 elements) — off-by-one bounds are the favourite trap in traversal questions.

Q98. Which character prints?

```
s = "PSEUDO"
PRINT s[3] // 0-based index
```

A) E **B)** D **C)** P **D)** U

Answer: D) U

Explanation: Counting from 0: 0=P, 1=S, 2=E, 3=U. So s[3] is **'U'**. Most wrong answers here come from starting the count at 1 — the neighbouring letters are planted as options for exactly that slip.

Q99. Given the array below (index starts at 0), what is printed?

```
arr = [54, 40, 45, 37, 15]
small = arr[0]
FOR i = 0 TO 4
  IF arr[i] < small THEN small = arr[i]
END FOR
PRINT small
```

A) 14 **B)** 17 **C)** 15 **D)** 16

Answer: C) 15

Explanation: The loop keeps the smallest element seen so far: comparisons leave the minimum, 15 → **15**. Confirm the loop covers indices 0..4 (all 5 elements) — off-by-one bounds are the favourite trap in traversal questions.

Q100. What is the output?

```
list = [72, 17, 77, 87, 21]
list[4] = list[1] + 5
PRINT list[4]
```

A) 22 **B)** 21 **C)** 25 **D)** 17

Answer: A) 22

Explanation: list[1] is 17, so the assignment stores 17 + 5 = **22** into position 4, overwriting 21. Assignments replace; they never add to the old cell value unless the cell itself appears on the right side.

Q101. Predict the printed element:

```
items = [89, 66, 85, 69, 88]
REVERSE the array in place
PRINT items[0]
```

A) 89 **B)** 88 **C)** 92 **D)** 90

Answer: B) 88

Explanation: Reversed, the array becomes [88, 69, 85, 66, 89]; index 0 now holds **88**, which originally sat at index 4. Shortcut: after reversal, new index k maps to old index (length – 1 – k) — apply the formula instead of rewriting the array.

Q102. What is the output?

```
items = [46, 77, 15, 19, 11]
PRINT items[3] + items[2]
```

A) 39 **B)** 31 **C)** 34 **D)** 19

Answer: C) 34

Explanation: Zero-based indexing: items[3] = 19 and items[2] = 15; their sum is **34**. Index n is the (n+1)th element — reciting that rule before answering prevents the shifted-by-one errors the options bait.

Q103. What substring is printed?

```
s = "PLACEMENT"
PRINT SUBSTRING(s, 3, 5) // start 3, end 5 EXCLUSIVE
```

A) EM **B)** CE **C)** CEM **D)** ACE

Answer: B) CE

Explanation: With an exclusive end, characters at indices 3..4 are taken: **"CE"** (2 characters). The inclusive-end reading gives "CEM" — sitting among the options, as always. Inclusive vs exclusive end is the entire battle in substring questions.

Q104. How many times does 'C' occur?

```
s = "COMPILE"
cnt = 0
FOR each ch IN s
  IF ch == 'C' THEN cnt = cnt + 1
END FOR
PRINT cnt
```

A) 0 **B)** 1 **C)** 2 **D)** 3

Answer: B) 1

Explanation: Scan "COMPILE" for 'C': it appears **1** time(s). Frequency loops compare each character to the target and increment a counter — mark the letters on the word as you count, because double-counting or skipping one letter is how the neighbouring options win.

Q105. Given the array below (index starts at 0), what is printed?

```
items = [21, 4, 13, 56, 59, 18]
total = 0
FOR i = 0 TO 5
  total = total + items[i]
END FOR
PRINT total
```

A) 169 **B)** 171 **C)** 168 **D)** 170

Answer: B) 171

Explanation: The loop adds every element: 21+4+13+56+59+18 → **171**. Confirm the loop covers indices 0..5 (all 6 elements) — off-by-one bounds are the favourite trap in traversal questions.

Q106. Predict the printed element:

```
marks = [83, 92, 33, 74, 93]
ROTATE LEFT by 2 position(s)
PRINT marks[4]
```

A) 94 **B)** 96 **C)** 90 **D)** 92

Answer: D) 92

Explanation: A left rotation by 2 moves the first 2 element(s) to the back: [33, 74, 93, 83, 92]. Index 4 then holds **92**. Mapping rule: new index k takes the old element at (k + 2) mod 5 — modular thinking beats rewriting under time pressure.

Q107. What is the output?

```
scores = [22, 20, 62, 53, 21]
scores[1] = scores[3] + 5
PRINT scores[1]
```

A) 53 **B)** 58 **C)** 63 **D)** 61

Answer: B) 58

Explanation: scores[3] is 53, so the assignment stores 53 + 5 = **58** into position 1, overwriting 20. Assignments replace; they never add to the old cell value unless the cell itself appears on the right side.

Q108. After sorting ascending, which value sits at the given index?

```
nums = [48, 45, 34, 81, 58, 22]
SORT nums in ascending order
PRINT nums[3]
```

A) 46 **B)** 52 **C)** 44 **D)** 48

Answer: D) 48

Explanation: Sorted: [22, 34, 45, 48, 58, 81]. Index 3 then holds **48** — the 4th smallest value. Post-sort index questions are rank questions in disguise; sort first, count second.

Q109. What does this display?

```
s = "KEYBOARD"
PRINT LENGTH(s)
```

A) 8 **B)** 10 **C)** 9 **D)** 6

Answer: A) 8

Explanation: LENGTH counts every character: KEYBOARD has **8** letters. Pair this with loops — 'FOR i = 0 TO LENGTH–1' visits all characters, and forgetting the –1 walks past the end, a bug the exam frequently encodes as an off-by-one option.

Q110. Given the array below (index starts at 0), what is printed?

```
nums = [23, 37, 22, 17, 8, 27]
big = nums[0]
FOR i = 0 TO 5
  IF nums[i] > big THEN big = nums[i]
END FOR
PRINT big
```

A) 37 **B)** 39 **C)** 35 **D)** 36

Answer: A) 37

Explanation: The loop tracks the largest element seen so far: comparisons leave the maximum, 37 → **37**. Confirm the loop covers indices 0..5 (all 6 elements) — off-by-one bounds are the favourite trap in traversal questions.

SECTION 4 — Recursion & Functions (Q111–Q130)

Q111. How many is the returned count?

```
FUNCTION H(n)
  IF n == 0 THEN RETURN 0
  RETURN 1 + H(n DIV 2)
END FUNCTION
PRINT H(13)
```

A) 3 **B)** 6 **C)** 4 **D)** 2

Answer: C) 4

Explanation: Each level halves n (integer division): 13 → 6 → 3 → 1 → 0, contributing 1 per step = **4**. This count is ■log■ n■ + 1 — the number of binary digits of 13 — the logarithmic pattern behind binary search complexity.

Q112. What value is returned by the outermost call?

```
FUNCTION T(n)
  IF n == 1 THEN RETURN 1
  RETURN n + T(n - 1)
END FUNCTION
PRINT T(8)
```

A) 36 **B)** 44 **C)** 33 **D)** 28

Answer: A) 36

Explanation: Each level adds its n: 8+7+6+5+4+3+2+1 = **36** — the sum 1..8, which the closed form n(n+1)/2 = 8×9/2 confirms instantly. Knowing that formula turns a 60-second trace into a 5-second calculation.

Q113. What is printed for this input?

```
FUNCTION D(n)
  IF n < 10 THEN RETURN n
  RETURN (n MOD 10) + D(n DIV 10)
END FUNCTION
PRINT D(894)
```

A) 19 **B)** 21 **C)** 23 **D)** 18

Answer: B) 21

Explanation: n MOD 10 peels the last digit and n DIV 10 drops it, so the function sums the digits of 894: 8 + 9 + 4 = **21**. MOD-10 / DIV-10 as 'last digit / remove last digit' is a pattern worth recognising on sight — it appears in reversals and palindromes too.

Q114. The classic two-branch recursion below returns what?

```
FUNCTION FIB(n)
  IF n < 2 THEN RETURN n
  RETURN FIB(n-1) + FIB(n-2)
END FUNCTION
PRINT FIB(6)
```

A) 6 **B)** 9 **C)** 8 **D)** 7

Answer: C) 8

Explanation: Build the Fibonacci ladder from the base: 0, 1, 1, 2, 3, 5, 8 — position 6 holds **8**. Never expand the recursion tree in an exam; grow the sequence upward from 0, 1 and read off the answer.

Q115. How many is the returned count?

```
FUNCTION H(n)
  IF n == 0 THEN RETURN 0
  RETURN 1 + H(n DIV 2)
END FUNCTION
PRINT H(68)
```

A) 7 **B)** 6 **C)** 9 **D)** 8

Answer: A) 7

Explanation: Each level halves n (integer division): 68 → 34 → 17 → 8 → 4 → 2 → 1 → 0, contributing 1 per step = **7**. This count is ■log■ n■ + 1 — the number of binary digits of 68 — the logarithmic pattern behind binary search complexity.

Q116. What value is returned by the outermost call?

```
FUNCTION ACC(n)
  IF n == 1 THEN RETURN 1
  RETURN n + ACC(n - 1)
END FUNCTION
PRINT ACC(9)
```

A) 54 **B)** 36 **C)** 45 **D)** 48

Answer: C) 45

Explanation: Each level adds its n: 9+8+7+6+5+4+3+2+1 = **45** — the sum 1..9, which the closed form n(n+1)/2 = 9×10/2 confirms instantly. Knowing that formula turns a 60-second trace into a 5-second calculation.

Q117. What is the result?

```
FUNCTION FUN(n)
  IF n == 0 THEN RETURN 2
  RETURN n * FUN(n - 1)
END FUNCTION
PRINT FUN(4)
```

A) 48 **B)** 24 **C)** 52 **D)** 72

Answer: A) 48

Explanation: Unwind the calls: FUN(4) = 4×FUN(3) = ... down to FUN(0) = 2, then multiply back up: 4×3×2×1×2 = **48**. This is n! doubled by the base value 2. Always find the base case first; it anchors the whole unwinding.

Q118. What is printed for this input?

```
FUNCTION D(n)
  IF n < 10 THEN RETURN n
  RETURN (n MOD 10) + D(n DIV 10)
END FUNCTION
PRINT D(385)
```

A) 13 **B)** 16 **C)** 18 **D)** 14

Answer: B) 16

Explanation: n MOD 10 peels the last digit and n DIV 10 drops it, so the function sums the digits of 385: 3 + 8 + 5 = **16**. MOD-10 / DIV-10 as 'last digit / remove last digit' is a pattern worth recognising on sight — it appears in reversals and palindromes too.

Q119. The classic two-branch recursion below returns what?

```
FUNCTION FIB(n)
  IF n < 2 THEN RETURN n
  RETURN FIB(n-1) + FIB(n-2)
END FUNCTION
PRINT FIB(8)
```

A) 20 **B)** 21 **C)** 24 **D)** 19

Answer: B) 21

Explanation: Build the Fibonacci ladder from the base: 0, 1, 1, 2, 3, 5, 8, 13, 21 — position 8 holds **21**. Never expand the recursion tree in an exam; grow the sequence upward from 0, 1 and read off the answer.

Q120. Evaluate the recursive call:

```
FUNCTION G(n)
  IF n == 0 THEN RETURN 1
  RETURN n * G(n - 1)
END FUNCTION
PRINT G(6)
```

A) 720 **B)** 726 **C)** 714 **D)** 1080

Answer: A) 720

Explanation: Unwind the calls: G(6) = 6×G(5) = ... down to G(0) = 1, then multiply back up: 6×5×4×3×2×1×1 = **720**. This is exactly n! (factorial). Always find the base case first; it anchors the whole unwinding.

Q121. What value is returned by the outermost call?

```
FUNCTION S(n)
  IF n == 1 THEN RETURN 1
  RETURN n + S(n - 1)
END FUNCTION
PRINT S(7)
```

A) 25 **B)** 31 **C)** 21 **D)** 28

Answer: D) 28

Explanation: Each level adds its n: 7+6+5+4+3+2+1 = **28** — the sum 1..7, which the closed form n(n+1)/2 = 7×8/2 confirms instantly. Knowing that formula turns a 60-second trace into a 5-second calculation.

Q122. The top-level call outputs:

```
FUNCTION G(n)
  IF n == 0 THEN RETURN 2
  RETURN n * G(n - 1)
END FUNCTION
PRINT G(5)
```

A) 360 **B)** 120 **C)** 240 **D)** 245

Answer: C) 240

Explanation: Unwind the calls: G(5) = 5×G(4) = ... down to G(0) = 2, then multiply back up: 5×4×3×2×1×2 = **240**. This is n! doubled by the base value 2. Always find the base case first; it anchors the whole unwinding.

Q123. How many is the returned count?

```
FUNCTION H(n)
  IF n == 0 THEN RETURN 0
  RETURN 1 + H(n DIV 2)
END FUNCTION
PRINT H(37)
```

A) 4 **B)** 5 **C)** 6 **D)** 8

Answer: C) 6

Explanation: Each level halves n (integer division): 37 → 18 → 9 → 4 → 2 → 1 → 0, contributing 1 per step = **6**. This count is ■log■ n■ + 1 — the number of binary digits of 37 — the logarithmic pattern behind binary search complexity.

Q124. The top-level call outputs:

```
FUNCTION CALC(n)
  IF n == 0 THEN RETURN 1
  RETURN n * CALC(n - 1)
END FUNCTION
PRINT CALC(4)
```

A) 12 **B)** 36 **C)** 24 **D)** 20

Answer: C) 24

Explanation: Unwind the calls: CALC(4) = 4×CALC(3) = ... down to CALC(0) = 1, then multiply back up: 4×3×2×1×1 = **24**. This is exactly n! (factorial). Always find the base case first; it anchors the whole unwinding.

Q125. The classic two-branch recursion below returns what?

```
FUNCTION FIB(n)
  IF n < 2 THEN RETURN n
  RETURN FIB(n-1) + FIB(n-2)
END FUNCTION
PRINT FIB(5)
```

A) 5 **B)** 8 **C)** 4 **D)** 3

Answer: A) 5

Explanation: Build the Fibonacci ladder from the base: 0, 1, 1, 2, 3, 5 — position 5 holds **5**. Never expand the recursion tree in an exam; grow the sequence upward from 0, 1 and read off the answer.

Q126. What is printed for this input?

```
FUNCTION D(n)
  IF n < 10 THEN RETURN n
  RETURN (n MOD 10) + D(n DIV 10)
END FUNCTION
PRINT D(811)
```

A) 12 **B)** 13 **C)** 7 **D)** 10

Answer: D) 10

Explanation: n MOD 10 peels the last digit and n DIV 10 drops it, so the function sums the digits of 811: 8 + 1 + 1 = **10**. MOD-10 / DIV-10 as 'last digit / remove last digit' is a pattern worth recognising on sight — it appears in reversals and palindromes too.

Q127. The classic two-branch recursion below returns what?

```
FUNCTION FIB(n)
  IF n < 2 THEN RETURN n
  RETURN FIB(n-1) + FIB(n-2)
END FUNCTION
PRINT FIB(7)
```

A) 11 **B)** 14 **C)** 13 **D)** 16

Answer: C) 13

Explanation: Build the Fibonacci ladder from the base: 0, 1, 1, 2, 3, 5, 8, 13 — position 7 holds **13**. Never expand the recursion tree in an exam; grow the sequence upward from 0, 1 and read off the answer.

Q128. What value is returned by the outermost call?

```
FUNCTION ACC(n)
  IF n == 1 THEN RETURN 1
  RETURN n + ACC(n - 1)
END FUNCTION
PRINT ACC(6)
```

A) 21 **B)** 18 **C)** 24 **D)** 28

Answer: A) 21

Explanation: Each level adds its n: 6+5+4+3+2+1 = **21** — the sum 1..6, which the closed form n(n+1)/2 = 6×7/2 confirms instantly. Knowing that formula turns a 60-second trace into a 5-second calculation.

Q129. What is the result?

```
FUNCTION F(n)
  IF n == 0 THEN RETURN 1
  RETURN n * F(n - 1)
END FUNCTION
PRINT F(5)
```

A) 120 **B)** 60 **C)** 115 **D)** 180

Answer: A) 120

Explanation: Unwind the calls: F(5) = 5×F(4) = ... down to F(0) = 1, then multiply back up: 5×4×3×2×1×1 = **120**. This is exactly n! (factorial). Always find the base case first; it anchors the whole unwinding.

Q130. What is printed for this input?

```
FUNCTION D(n)
  IF n < 10 THEN RETURN n
  RETURN (n MOD 10) + D(n DIV 10)
END FUNCTION
PRINT D(615)
```

A) 15 **B)** 14 **C)** 10 **D)** 12

Answer: D) 12

Explanation: n MOD 10 peels the last digit and n DIV 10 drops it, so the function sums the digits of 615: 6 + 1 + 5 = **12**. MOD-10 / DIV-10 as 'last digit / remove last digit' is a pattern worth recognising on sight — it appears in reversals and palindromes too.

SECTION 5 — Operators & Expression Evaluation (Q131–Q150)

Q131. Evaluate:

```
PRINT (8 + 7) * 8
```

A) 120 **B)** 123 **C)** 122 **D)** 64

Answer: A) 120

Explanation: Apply precedence — parentheses override precedence: 8+7 = 15, then ×8 — giving **120**. The left-to-right misreading produces 64, predictably present in the options. BODMAS/PEMDAS questions are pure discipline, zero difficulty.

Q132. What is printed?

```
PRINT 19 DIV 3
```

A) 9 **B)** 7 **C)** 4 **D)** 6

Answer: D) 6

Explanation: Integer division discards the remainder: 19 ÷ 3 = 6.33, keep the whole part **6**. The rounded value 6 is the trap when the fraction exceeds .5.

Q133. What FINAL value of q prints?

```
q = 9
d = 8
q = q + d
q = q + d
PRINT q
```

A) 17 **B)** 25 **C)** 16 **D)** 34

Answer: B) 25

Explanation: Sequential updates: after line 3, q = 9+8 = 17; line 4 adds d (still 8) again → **25**. d never changed — re-reading which variable each line actually modifies is the whole skill in sequencing questions.

Q134. What FINAL value of ans prints?

```
ans = 5
c = 10
ans = ans + c
ans = ans + c
PRINT ans
```

A) 25 **B)** 35 **C)** 15 **D)** 20

Answer: A) 25

Explanation: Sequential updates: after line 3, ans = 5+10 = 15; line 4 adds c (still 10) again → **25**. c never changed — re-reading which variable each line actually modifies is the whole skill in sequencing questions.

Q135. What is printed?

```
PRINT (95 DIV 8) + (95 MOD 8)
```

A) 17 **B)** 19 **C)** 21 **D)** 18

Answer: D) 18

Explanation: Quotient 11 plus remainder 7 = **18**. DIV and MOD are partners: quotient×divisor + remainder rebuilds the original number — use that identity to self-check.

Q136. Evaluate:

```
PRINT 2 + 6 * 8
```

A) 64 **B)** 50 **C)** 48 **D)** 53

Answer: B) 50

Explanation: Apply precedence — multiplication binds first: 6×8 = 48, then + 2 — giving **50**. The left-to-right misreading produces 64, predictably present in the options. BODMAS/PEMDAS questions are pure discipline, zero difficulty.

Q137. What is printed?

```
PRINT 39 MOD 4
```

A) 1 **B)** 3 **C)** 2 **D)** 4

Answer: B) 3

Explanation: 4 × 9 = 36, and 39 – 36 = **3** is the remainder. MOD results always sit in 0..3 — any option outside that range is instantly wrong.

Q138. What is printed?

```
PRINT 43 DIV 4
```

A) 10 **B)** 11 **C)** 9 **D)** 12

Answer: A) 10

Explanation: Integer division discards the remainder: 43 ÷ 4 = 10.75, keep the whole part **10**. The rounded value 11 is the trap when the fraction exceeds .5.

Q139. What does the condition evaluate to?

```
PRINT (6 > 3) OR (14 < 10)
```

A) TRUE **B)** FALSE **C)** 1 **D)** Error

Answer: A) TRUE

Explanation: First comparison: 6 > 3 is True; second: 14 < 10 is False. OR therefore yields **TRUE** — OR needs just one TRUE. Evaluate each side fully before applying the connective; mixing the two steps is where errors creep in.

Q140. Evaluate:

```
PRINT 9 + 8 * 9
```

A) 83 **B)** 79 **C)** 84 **D)** 81

Answer: D) 81

Explanation: Apply precedence — multiplication binds first: 8×9 = 72, then + 9 — giving **81**. The left-to-right misreading produces 153, predictably present in the options. BODMAS/PEMDAS questions are pure discipline, zero difficulty.

Q141. Evaluate:

```
PRINT 7 * 8 - 7 * 2
```

A) 44 **B)** 42 **C)** 14 **D)** 45

Answer: B) 42

Explanation: Apply precedence — both products evaluate before the subtraction: 56 – 14 — giving **42**. The left-to-right misreading produces 14, predictably present in the options. BODMAS/PEMDAS questions are pure discipline, zero difficulty.

Q142. Evaluate:

```
PRINT (2 + 9) * 6
```

A) 66 **B)** 56 **C)** 68 **D)** 69

Answer: A) 66

Explanation: Apply precedence — parentheses override precedence: 2+9 = 11, then ×6 — giving **66**. The left-to-right misreading produces 56, predictably present in the options. BODMAS/PEMDAS questions are pure discipline, zero difficulty.

Q143. What is printed?

```
PRINT (44 DIV 9) + (44 MOD 9)
```

A) 12 **B)** 11 **C)** 13 **D)** 10

Answer: A) 12

Explanation: Quotient 4 plus remainder 8 = **12**. DIV and MOD are partners: quotient×divisor + remainder rebuilds the original number — use that identity to self-check.

Q144. Evaluate:

```
PRINT 6 * 4 - 3 * 2
```

A) 18 **B)** 16 **C)** 12 **D)** 21

Answer: A) 18

Explanation: Apply precedence — both products evaluate before the subtraction: 24 – 6 — giving **18**. The left-to-right misreading produces 12, predictably present in the options. BODMAS/PEMDAS questions are pure discipline, zero difficulty.

Q145. What does the condition evaluate to?

```
PRINT (6 > 9) OR (14 < 10)
```

A) TRUE **B)** FALSE **C)** 1 **D)** Error

Answer: B) FALSE

Explanation: First comparison: 6 > 9 is False; second: 14 < 10 is False. OR therefore yields **FALSE** — OR needs just one TRUE. Evaluate each side fully before applying the connective; mixing the two steps is where errors creep in.

Q146. What does the condition evaluate to?

```
PRINT (18 > 15) AND (8 < 4)
```

A) TRUE **B)** FALSE **C)** 1 **D)** Error

Answer: B) FALSE

Explanation: First comparison: 18 > 15 is True; second: 8 < 4 is False. AND therefore yields **FALSE** — AND demands both TRUE. Evaluate each side fully before applying the connective; mixing the two steps is where errors creep in.

Q147. The no-temp arithmetic swap runs below. What does y print?

```
x = 12 ; y = 8
x = x + y
y = x - y
x = x - y
PRINT y
```

A) 12 **B)** 17 **C)** 8 **D)** 10

Answer: A) 12

Explanation: Trace: x becomes 20; then y = 20–8 = 12; then x = 20–12 = 8. The values SWAP, so y prints **12**. Like the XOR version, learn this as a named pattern — 'add-subtract swap' — and skip the per-line grind.

Q148. The no-temp arithmetic swap runs below. What does a print?

```
val = 3 ; a = 12
val = val + a
a = val - a
val = val - a
PRINT a
```

A) 3 **B)** 1 **C)** 8 **D)** 12

Answer: A) 3

Explanation: Trace: val becomes 15; then a = 15–12 = 3; then val = 15–3 = 12. The values SWAP, so a prints **3**. Like the XOR version, learn this as a named pattern — 'add-subtract swap' — and skip the per-line grind.

Q149. What is printed?

```
PRINT 94 MOD 9
```

A) 5 **B)** 3 **C)** 4 **D)** 6

Answer: C) 4

Explanation: 9 × 10 = 90, and 94 – 90 = **4** is the remainder. MOD results always sit in 0..8 — any option outside that range is instantly wrong.

Q150. What does the condition evaluate to?

```
PRINT (10 > 7) AND (5 < 9)
```

A) TRUE **B)** FALSE **C)** 1 **D)** Error

Answer: A) TRUE

Explanation: First comparison: 10 > 7 is True; second: 5 < 9 is True. AND therefore yields **TRUE** — AND demands both TRUE. Evaluate each side fully before applying the connective; mixing the two steps is where errors creep in.

SECTION 6 — OOPs Concepts (Q151–Q200)

Q151. Encapsulation in OOP means:

A) Creating multiple classes **B)** Bundling data and the methods operating on it, restricting direct access **C)** Deleting unused objects **D)** Copying one object into another

Answer: B) Bundling data and the methods operating on it, restricting direct access

Explanation: Encapsulation wraps an object's state (fields) together with its behaviour (methods) and hides internals behind access modifiers — outsiders interact only through the public interface. A bank-account class exposing deposit()/withdraw() while keeping balance private is the textbook illustration. 'Data hiding + bundling' is the two-word summary examiners accept.

Q152. Which OOP pillar lets a Dog class reuse the code of an Animal class?

A) Abstraction **B)** Polymorphism **C)** Inheritance **D)** Encapsulation

Answer: C) Inheritance

Explanation: Inheritance creates an is-a relationship: Dog EXTENDS Animal and automatically owns Animal's non-private members, adding or overriding what it needs. Code reuse plus hierarchical modelling is its purpose. The keyword differs by language — extends (Java), : (C#/C++) — but the concept question stays identical.

Q153. Polymorphism literally means:

A) One class only **B)** Many forms — the same interface behaving differently for different types **C)** Hidden data **D)** Fast execution

Answer: B) Many forms — the same interface behaving differently for different types

Explanation: Poly (many) + morph (form): one method call, many behaviours. animal.speak() barks for a Dog and meows for a Cat even through an Animal reference — the runtime picks the right version. Compile-time (overloading) and runtime (overriding) are its two flavours, each with its own exam questions.

Q154. Abstraction focuses on:

A) Showing essential features while hiding implementation details **B)** Making all members public **C)** Copying objects **D)** Speeding up loops

Answer: A) Showing essential features while hiding implementation details

Explanation: Abstraction answers WHAT an object does, not HOW. A driver uses brake() without knowing hydraulics; code uses List.sort() without reading the algorithm. Abstract classes and interfaces are the language tools for it. Abstraction hides COMPLEXITY, encapsulation hides DATA — exams test that exact contrast.

Q155. A class is best described as:

A) A running program **B)** A blueprint/template from which objects are created **C)** A variable **D)** A database table

Answer: B) A blueprint/template from which objects are created

Explanation: The class defines structure (fields) and behaviour (methods); objects are its concrete instances at runtime. One Car class, many car objects with different colours and speeds. Blueprint-to-building is the analogy that answers every phrasing of this definitional staple.

Q156. An object is:

A) A copy of the compiler **B)** An instance of a class occupying memory with its own state **C)** A function **D)** A comment

Answer: B) An instance of a class occupying memory with its own state

Explanation: Instantiation (e.g., new Car()) allocates memory and produces an object whose field values form its state, manipulated via its methods. Two objects of one class are independent — changing one's speed leaves the other untouched. Class = design, object = reality: the paired definition to keep ready.

Q157. A constructor is special because it:

A) Returns int by default **B)** Runs automatically when an object is created and shares the class's name **C)** Must be private **D)** Can never take parameters

Answer: B) Runs automatically when an object is created and shares the class's name

Explanation: Constructors initialise new objects: same name as the class (or __init__ in Python), NO return type (not even void), invoked by 'new'. They absolutely can take parameters — parameterised constructors are the norm. 'No return type' and 'auto-invoked' are the two properties questions probe.

Q158. If no constructor is written, Java/C# provide:

A) A compile error **B)** A default no-argument constructor automatically **C)** A random constructor **D)** Nothing — objects can't be made

Answer: B) A default no-argument constructor automatically

Explanation: The compiler silently supplies a public no-arg default constructor that leaves fields at their default values. The twist: write ANY constructor yourself and the freebie disappears — then new X() fails unless you add the no-arg version back. That disappearing-default rule is the favourite follow-up question.

Q159. Method OVERLOADING is:

A) Same method name, different parameter lists, in the same class **B)** Same signature in parent and child **C)** Renaming a method **D)** Calling a method twice

Answer: A) Same method name, different parameter lists, in the same class

Explanation: Overloading = one name, multiple parameter patterns (count, types, order) resolved at COMPILE time — print(int), print(String), print(double). Return type alone cannot distinguish overloads. It's compile-time (static) polymorphism; its sibling overriding is the runtime kind — the pair is tested together constantly.

Q160. Method OVERRIDING requires:

A) Different method names **B)** A child class redefining a parent method with the same signature **C)** Static methods **D)** Private methods

Answer: B) A child class redefining a parent method with the same signature

Explanation: Overriding: subclass provides its own body for an inherited method — identical name and parameters — and the runtime dispatches to the child version through a parent reference (dynamic binding). Private and static methods can't be overridden (they're not polymorphic). Same-signature-different-class is the fingerprint distinguishing it from overloading.

Q161. Which access modifier makes a member visible ONLY within its own class?

A) public **B)** protected **C)** private **D)** default

Answer: C) private

Explanation: private confines access to the declaring class itself — the enforcement mechanism of encapsulation. protected extends to subclasses (and same package in Java); public opens to all; default (package-private in Java) covers the package. The visibility ladder private → default → protected → public is a guaranteed one-marker.

Q162. The 'protected' modifier in Java allows access from:

A) Anywhere **B)** Same class only **C)** Same package plus subclasses (even in other packages) **D)** Only main()

Answer: C) Same package plus subclasses (even in other packages)

Explanation: protected = package access PLUS inheritance access: subclasses anywhere may use the member, as can any same-package class. It sits between default and public in openness. The 'subclass in a different package' scenario is precisely what separates protected from default — and what the exam asks.

Q163. A static member of a class:

A) Belongs to each object separately **B)** Belongs to the class itself, shared by all instances **C)** Cannot be accessed **D)** Is always constant

Answer: B) Belongs to the class itself, shared by all instances

Explanation: static means ONE copy at class level: a static counter incremented in the constructor counts all objects ever made, and Math.max() runs without any object. static ≠ constant (that's final/const). 'Shared by all instances, accessible via the class name' is the definition to lock in.

Q164. What is the output concept: a parent reference holding a child object calls an overridden method. Whose version runs?

A) Parent's version **B)** Child's version (runtime/dynamic dispatch) **C)** Both **D)** Compile error

Answer: B) Child's version (runtime/dynamic dispatch)

Explanation: Animal a = new Dog(); a.speak() → the DOG's speak runs. The reference type decides what you MAY call; the object's actual type decides WHICH implementation executes — that's dynamic method dispatch, the engine of runtime polymorphism, and the single most-tested OOPs behaviour in these exams.

Q165. An abstract class:

A) Can be instantiated directly **B)** May contain abstract (bodyless) methods and cannot be instantiated **C)** Has only static members **D)** Is always final

Answer: B) May contain abstract (bodyless) methods and cannot be instantiated

Explanation: Abstract classes are incomplete blueprints: they can mix concrete methods with abstract declarations that subclasses MUST implement, and 'new AbstractThing()' is illegal. They can have constructors (called via subclass chains) — a subtle truth exams exploit. Incomplete-by-design is the core idea.

Q166. An interface (Java) traditionally contains:

A) Method signatures (contracts) that implementing classes must fulfil **B)** Only private fields **C)** Executable main methods only **D)** Database queries

Answer: A) Method signatures (contracts) that implementing classes must fulfil

Explanation: An interface declares WHAT a class can do — a pure contract; classes 'implement' it and supply bodies. Fields are implicitly public static final, and since Java 8 default/static methods with bodies are allowed (a common 'modern' question). A class implements MANY interfaces, sidestepping single-inheritance limits.

Q167. Why do Java and C# forbid multiple class inheritance?

A) Memory cost **B)** The diamond problem — ambiguity when two parents define the same method **C)** Legal reasons **D)** It's allowed actually

Answer: B) The diamond problem — ambiguity when two parents define the same method

Explanation: If B and C both inherit A and override the same method, a D extending both wouldn't know which version to use — the DIAMOND ambiguity. Java/C# avoid it by allowing one superclass but many interfaces; C++ permits multiple inheritance and resolves diamonds with virtual inheritance. Name 'diamond problem' — that phrase is the expected answer.

Q168. The 'this' keyword refers to:

A) The parent class **B)** The current object whose method is executing **C)** A static context **D)** The main thread

Answer: B) The current object whose method is executing

Explanation: 'this' is a self-reference: this.name = name distinguishes the field from a same-named parameter; this() chains to another constructor of the SAME class. It doesn't exist in static contexts (no current object there) — a corner the exam pokes. Current-object pointer, in three words.

Q169. The 'super' keyword is used to:

A) Create objects faster **B)** Access the parent class's members or constructor from a child **C)** Make methods public **D)** Stop inheritance

Answer: B) Access the parent class's members or constructor from a child

Explanation: super.method() calls the parent's version even when overridden; super(args) invokes the parent constructor and, when used, must be the FIRST statement in the child constructor — the ordering rule questions single out. this = my class; super = one level up: keep the pair straight.

Q170. Which concept does 'IS-A' vs 'HAS-A' distinguish?

A) Overloading vs overriding **B)** Inheritance vs composition **C)** Class vs object **D)** Public vs private

Answer: B) Inheritance vs composition

Explanation: IS-A → inheritance (Car IS-A Vehicle → extend). HAS-A → composition (Car HAS-A Engine → keep an Engine field). Design questions describe a relationship and ask which to use; modern guidance even prefers composition where either fits ('favour composition over inheritance') — a phrase worth recognising.

Q171. Data hiding is achieved primarily through:

A) Loops **B)** Private fields with public getter/setter methods **C)** Comments **D)** Inheritance

Answer: B) Private fields with public getter/setter methods

Explanation: Make state private, expose controlled access via getters/setters — the setter can validate (reject negative age), the getter can format. Direct field access would bypass every rule. This getter/setter pattern is encapsulation IN PRACTICE and the implementation-level answer when the conceptual one (encapsulation) isn't among the options.

Q172. A 'final' class in Java:

A) Runs last **B)** Cannot be subclassed **C)** Has no methods **D)** Is abstract

Answer: B) Cannot be subclassed

Explanation: final on a class bans inheritance (String is the famous example); final on a METHOD bans overriding; final on a VARIABLE makes it single-assignment (a constant). Three targets, three effects — questions name one and offer the other two as distractors. final and abstract are opposites and can never combine.

Q173. Constructor overloading means:

A) Two classes share a constructor **B)** A class defines multiple constructors with different parameter lists **C)** Constructors return values **D)** Child overrides parent constructor

Answer: B) A class defines multiple constructors with different parameter lists

Explanation: A class may offer several construction recipes — Box(), Box(size), Box(w, h) — chosen by argument pattern, often chained via this(...). Constructors are never OVERRIDDEN (they aren't inherited), which kills option D and forms a neat true/false trap the exam reuses.

Q174. What does 'new' do in object creation?

A) Declares a variable **B)** Allocates memory for the object and triggers its constructor **C)** Deletes old objects **D)** Imports a class

Answer: B) Allocates memory for the object and triggers its constructor

Explanation: new = allocate on the heap + run the constructor + hand back a reference. Declaration alone (Car c;) creates only a reference holding nothing — using it uninitialised is the null-pointer scenario. The allocation-then-construct sequence is what the question checks you can articulate.

Q175. Garbage collection in Java/C# refers to:

A) Deleting source files **B)** Automatic reclamation of memory held by unreachable objects **C)** Cleaning the console **D)** Removing comments

Answer: B) Automatic reclamation of memory held by unreachable objects

Explanation: The runtime tracks reachability; objects no reference can reach become garbage and their memory is reclaimed automatically — no manual free/delete as in C/C++. You can request collection (System.gc()) but never command it deterministically. 'Automatic + unreachable' are the two words the correct option must contain.

Q176. Which is TRUE about a static method?

A) It can directly use instance variables **B)** It is called on the class and cannot directly access non-static members **C)** It must return void **D)** It runs before main

Answer: B) It is called on the class and cannot directly access non-static members

Explanation: Static methods have NO 'this' — there's no object — so touching instance fields directly is a compile error; they work with parameters and static state. main() itself is static for exactly this reason: it must run before any object exists. The no-this consequence is the tested logic.

Q177. Runtime polymorphism in Java is achieved by:

A) Method overloading **B)** Method overriding with upcasting (parent reference, child object) **C)** Constructors **D)** Static blocks

Answer: B) Method overriding with upcasting (parent reference, child object)

Explanation: The recipe: override a method in the child, hold the child object in a PARENT-type reference, call the method — dispatch resolves at runtime to the child. Overloading resolves at compile time, making option A the perpetual wrong-but-tempting choice. 'Overriding + upcasting' is the full correct phrase.

Q178. Which pair correctly matches binding types?

A) Overloading → runtime, Overriding → compile-time **B)** Overloading → compile-time (early), Overriding → runtime (late) **C)** Both compile-time **D)** Both runtime

Answer: B) Overloading → compile-time (early), Overriding → runtime (late)

Explanation: Overloading is resolved by the compiler from the argument types — EARLY/static binding. Overriding is resolved by the object's actual class during execution — LATE/dynamic binding. The reversed pairing (option A) is the trap; anchor it as 'overRIDING happens at RUN time' — the R's match.

Q179. A local variable inside a method differs from an instance variable because it:

A) Gets a default value automatically **B)** Must be initialised before use and dies when the method ends **C)** Is shared across objects **D)** Is always public

Answer: B) Must be initialised before use and dies when the method ends

Explanation: Locals live on the stack for the method's duration and the compiler REFUSES to read them uninitialised. Instance fields live with the object and DO receive defaults (0, false, null). Lifetime + default-value are the two contrasts; either alone identifies the correct option.

Q180. What is a 'getter' method?

A) A method that deletes a field **B)** A public method returning the value of a private field **C)** A constructor **D)** A static initialiser

Answer: B) A public method returning the value of a private field

Explanation: getBalance() { return balance; } — read access to hidden state, conventionally named getX (isX for booleans). Its partner setter writes with validation. Together they form the JavaBean property pattern that frameworks and exams both rely on. Read-door to private data: that's a getter.

Q181. Composition is preferred over inheritance when:

A) The relationship is 'is-a' **B)** You only need to USE another class's functionality (has-a), not BE that class **C)** Methods must be overridden **D)** Classes are final

Answer: B) You only need to USE another class's functionality (has-a), not BE that class

Explanation: If Car merely NEEDS an Engine, embed one — composition keeps coupling low and lets parts swap at runtime. Inheriting Engine would falsely claim Car IS an Engine and drag in its whole interface. Relationship-verb analysis (is vs has) answers every design-choice question of this family.

Q182. What happens when a subclass constructor runs?

A) Only the subclass constructor executes **B)** The parent constructor executes first (implicitly or via super), then the child's body **C)** Constructors merge **D)** Parent fields stay uninitialised

Answer: B) The parent constructor executes first (implicitly or via super), then the child's body

Explanation: Construction is top-down: super's constructor completes before the child's body starts (an implicit super() is inserted if you don't write one). Print statements in both constructors make the order visible — 'Parent' then 'Child' — the exact output-prediction question these exams print.

Q183. The 'instanceof' operator (or 'is' in C#) checks:

A) If a variable is initialised **B)** Whether an object is an instance of a given class/interface at runtime **C)** Whether two classes are equal **D)** Method existence

Answer: B) Whether an object is an instance of a given class/interface at runtime

Explanation: obj instanceof Dog returns true if obj's actual runtime type is Dog or a subtype — the safe test BEFORE downcasting to avoid ClassCastException. It examines objects, not references or classes-vs-classes. 'Runtime type check before casting' is the use-case phrasing to match.

Q184. Upcasting vs downcasting:

A) Both need explicit casts **B)** Upcasting (child→parent) is implicit and safe; downcasting (parent→child) is explicit and can fail at runtime **C)** Downcasting is always safe **D)** Neither compiles

Answer: B) Upcasting (child→parent) is implicit and safe; downcasting (parent→child) is explicit and can fail at runtime

Explanation: Every Dog IS an Animal, so Animal a = dog needs no cast. The reverse claims MORE specific knowledge — Dog d = (Dog) a — compiles with the explicit cast but explodes at runtime if a actually holds a Cat. Safe-up, risky-down, guarded by instanceof: the three-part rule.

Q185. A pure virtual function in C++ corresponds to what in Java?

A) A static method **B)** An abstract method **C)** A final method **D)** A constructor

Answer: B) An abstract method

Explanation: C++'s 'virtual void draw() = 0;' declares behaviour without body, forcing derived classes to implement — exactly Java's abstract method inside an abstract class. Cross-language terminology mapping (virtual/pure-virtual ↔ overridable/abstract) shows up because the exam permits multiple languages in the coding round.

Q186. Which statement about constructors and inheritance is TRUE?

A) Constructors are inherited by subclasses **B)** Constructors are NOT inherited; subclasses call them via super() **C)** Only default constructors are inherited **D)** Static constructors are inherited

Answer: B) Constructors are NOT inherited; subclasses call them via super()

Explanation: A subclass never inherits parent constructors — it must define its own and may CHAIN to the parent through super(args). This is why adding only a parameterised constructor to a parent forces children to call it explicitly. 'Not inherited, but chainable' resolves several true/false variants at once.

Q187. Encapsulation vs Abstraction — the cleanest distinction:

A) They are identical **B)** Encapsulation hides DATA (implementation-level); abstraction hides COMPLEXITY (design-level) **C)** Abstraction uses private fields **D)** Encapsulation needs interfaces

Answer: B) Encapsulation hides DATA (implementation-level); abstraction hides COMPLEXITY (design-level)

Explanation: Encapsulation is the mechanism (private fields, controlled access); abstraction is the mindset (expose essential operations, bury the how). A List interface is abstraction; ArrayList's private element array is encapsulation. Exams ask this contrast directly — memorise the data-vs-complexity phrasing.

Q188. What is an anonymous object?

A) An object with no class **B)** An object created without storing its reference, e.g., new Car().start() **C)** A null reference **D)** A deleted object

Answer: B) An object created without storing its reference, e.g., new Car().start()

Explanation: new Car().start() builds an object, uses it once, and abandons it — no variable holds it, so it's garbage-eligible immediately after. Useful for one-shot calls. It very much HAS a class; only the reference name is missing, which is the misconception option A encodes.

Q189. Why is main() declared static in Java?

A) Tradition **B)** So the JVM can invoke it without creating any object first **C)** To make it faster **D)** Because it returns void

Answer: B) So the JVM can invoke it without creating any object first

Explanation: Before main runs, zero objects exist — a non-static main would need an instance the JVM has no way to build. static lets the JVM call ClassName.main(args) directly. The reasoning chain (no objects yet → must be class-level) is what the question wants, not the keyword alone.

Q190. Which is an example of compile-time polymorphism?

A) Overriding toString() **B)** Method overloading like add(int,int) and add(double,double) **C)** Dynamic dispatch **D)** instanceof checks

Answer: B) Method overloading like add(int,int) and add(double,double)

Explanation: The compiler selects the right add() from the argument types before the program ever runs — early binding. Overriding, dispatch, and instanceof all involve RUNTIME behaviour. When an exam shows two same-named methods with different parameters in ONE class, it's naming overloading/compile-time — that pattern-recognition is the skill.

Q191. A class implementing an interface must:

A) Extend it **B)** Provide bodies for all its abstract methods (or itself be declared abstract) **C)** Be final **D)** Have a main method

Answer: B) Provide bodies for all its abstract methods (or itself be declared abstract)

Explanation: implements is a promise: every abstract method of the interface gets an implementation, otherwise the class must be marked abstract and pass the duty down. Default methods (Java 8+) come pre-implemented and are optional to override. Contract-fulfilment-or-stay-abstract is the rule being tested.

Q192. Object state and behaviour correspond to:

A) Methods and classes **B)** Fields (variables) and methods respectively **C)** Constructors and destructors **D)** Packages and imports

Answer: B) Fields (variables) and methods respectively

Explanation: State = the current values of an object's fields (speed=60, colour=red); behaviour = what it can do, its methods (accelerate(), brake()). Identity is the third classical property (each object is distinct even with equal state). State/behaviour/identity — the trio behind many definition questions.

Q193. Which keyword prevents a METHOD from being overridden?

A) static **B)** final **C)** private **D)** Both final and private effectively

Answer: D) Both final and private effectively

Explanation: final explicitly forbids overriding. private methods aren't visible to subclasses, so they can't be overridden either (a same-signature child method is a NEW method, not an override). Exams accept 'final' alone when it's the only candidate, but the combined option is the technically complete one — read all four before answering.

Q194. What is method hiding?

A) Deleting methods **B)** A static method in a child with the same signature as a static parent method — resolved by reference type **C)** Private inheritance **D)** Using final

Answer: B) A static method in a child with the same signature as a static parent method — resolved by reference type

Explanation: Static methods don't participate in dynamic dispatch; redefining one in a subclass HIDES the parent version, and calls resolve by the REFERENCE type at compile time. Parent p = new Child(); p.staticMethod() runs the PARENT's — the opposite of overriding's behaviour, and a wickedly effective exam question.

Q195. An aggregation relationship differs from composition because:

A) Aggregation is stronger **B)** In aggregation the part can exist independently of the whole (Dept–Teacher); in composition it cannot (House–Room) **C)** Composition uses interfaces **D)** They're synonyms

Answer: B) In aggregation the part can exist independently of the whole (Dept–Teacher); in composition it cannot (House–Room)

Explanation: Both are HAS-A, differing in lifecycle coupling: destroy the university and its teachers still exist (aggregation, hollow diamond in UML); demolish the house and its rooms are gone (composition, filled diamond). 'Can the part outlive the whole?' — answering that one question classifies any scenario.

Q196. Which is TRUE about interfaces vs abstract classes (classic Java)?

A) Abstract classes support multiple inheritance of type; interfaces don't **B)** A class extends ONE abstract class but implements MANY interfaces **C)** Interfaces can have instance fields **D)** Abstract classes can't have constructors

Answer: B) A class extends ONE abstract class but implements MANY interfaces

Explanation: Single class inheritance, unlimited interface implementation — the structural difference every comparison table leads with. Abstract classes DO have constructors and instance fields; classic interfaces have neither (fields are public static final). Options C and D invert those facts, making elimination straightforward if the table is memorised.

Q197. What does tight coupling mean, and why is it discouraged?

A) Classes compiled together **B)** Classes depending heavily on each other's internals, making change and testing hard **C)** Using many interfaces **D)** Fast method calls

Answer: B) Classes depending heavily on each other's internals, making change and testing hard

Explanation: Tightly coupled classes know too much about each other — modify one and the other breaks; unit-testing in isolation becomes impossible. OOP counters it with encapsulation, interfaces, and dependency injection (program to abstractions). 'Hard to change, hard to test' is the consequence pair the correct option must state.

Q198. The Single Responsibility Principle (S of SOLID) states:

A) A class should have only one reason to change — one job **B)** Classes must have one method **C)** Only one class per file **D)** One object per class

Answer: A) A class should have only one reason to change — one job

Explanation: SRP: each class owns exactly one responsibility; an Invoice class that also prints and saves itself has three reasons to change and should be split. It's about responsibilities, not method counts or file counts — the literal-sounding distractors. SOLID's first letter is the one exams pick most.

Q199. Shallow copy vs deep copy of an object:

A) Identical results always **B)** Shallow copies share referenced sub-objects; deep copy duplicates the entire object graph **C)** Deep copy is faster **D)** Shallow copy is illegal

Answer: B) Shallow copies share referenced sub-objects; deep copy duplicates the entire object graph

Explanation: A shallow copy duplicates the top object but both copies point to the SAME inner objects — mutate a shared list through one and the other sees it. Deep copy clones recursively, achieving full independence at higher cost. The shared-mutation surprise is the scenario exam questions dramatise.

Q200. What is a singleton class?

A) A class with one method **B)** A class designed so that only ONE instance can ever exist, with global access to it **C)** A final class **D)** An abstract class

Answer: B) A class designed so that only ONE instance can ever exist, with global access to it

Explanation: Singleton = private constructor + a static instance + a public static getInstance() returning that sole object — used for loggers, configuration, connection pools. 'Only one object, globally reachable' is the definition; the private constructor detail is the mechanism follow-up the exam asks next.

You have completed QB-3 (200/200). Next in the series: QB-3 Pseudocode + OOPs, QB-4 Networking/Security/Cloud, QB-5 CS Fundamentals, QB-6 Coding DSA, QB-7 SQL, QB-8 Frontend — and 10 Full-Length Mock Tests.
Made by Online Learning.